

[L0-L1 Core Essence & Design Logic]

L0 Essence: An in-memory key-value database engine that executes operations via a single-threaded event loop, maps datasets to compact CPU-cache-aligned representations, and ensures persistence and clustering via COW snapshots and Gossip shard routing.

L1 Logic:

- Single-Threaded Event Loop (M1)** offloads socket I/O but executes command logic sequentially on `ae.c`, bypassing thread context switching and lock overheads.
- Compact Dynamic Structures (M2)** leverage SDS, Ziplist/Listpack, and Skiplists to maximize cache hits and perform progressive hashing via double hash tables.
- Copy-on-Write and Append-Only Logs (M3)** split background writes from main processes using child process page table sharing (`fork()`) and sequential WAL execution.
- Approximated Eviction and Gossip Clustering (M4-M6)** use randomized key sampling to replace global LRU locks and map datasets across 16384 slots without central metadata nodes.

[L2 Redis Kernel Data Flow Topology]

• [RESP Client Connection] → [`ae.c` Event Loop Listener] → [SDS Query Buffer Load] → [Dict Command Hash Match] → [Memory Address Seek & Progressive Rehash] → [AOF Append / RDB COW Fork] → [Master Backlog Broadcast] → [Cluster Slot Redirect]

[M1.1] `ae.c` Single-Threaded Event Loop

- **Event Merging:** The core loop in `aeProcessEvents` coordinates both network events (socket reads/writes) and timer tasks (e.g., key expiration, clustering heartbeat).
- **Zero Locking:** By running entirely on a single thread, Redis avoids thread switching, mutex locking, and cache line invalidations caused by thread scheduling.

[M1.2] I/O Multiplexing Abstraction

- **OS Interface Wrapper:** Redis abstracts operating system non-blocking select interfaces via conditional compiles (`ae_select.c`, `ae_epoll.c`, `ae_kqueue.c`).
- **Platform Selection:** Selects the fastest interface (`epoll` on Linux, `kqueue` on BSD) to support $O(1)$ event callbacks on thousands of sockets.

[M1.3] Client Connection Buffers

- **Buffer Controls:** Client connection descriptors allocate a 16KB query read buffer that expands to a 1GB safety limit, dropping connections exceeding this limit.
- **Write Response Buffers:** Outbound payloads are formatted into client static buffers or appended to queue link lists before being flushed during socket write events.

[M1.4] RESP Protocol Parser

- **Binary-Safe Format:** RESP (Redis Serialization Protocol) uses prefix codes (e.g., `+` for strings, `$` for bulk strings, `:` for integers) to parse streams.
- **Low Overhead:** The parser splits data payloads without JSON/XML parser allocations, ensuring safe execution with binary strings using length headers.

[M2.1] SDS (Simple Dynamic String)

- **Length Trackers:** SDS structures store current length `len`, allocated capacity `alloc`, and header metadata `flags` to provide $O(1)$ length operations.
- **Overflow Protection:** SDS allocation monitors bounds before write operations, expanding capacity (doubling if under 1MB, adding 1MB if over) to handle binary payloads.

[M2.2] Diet Progressive Rehash

- **Incremental Hashing:** Redis dictionaries manage hash collisions using two tables, `ht[0]` and `ht[1]`. If table size exceeds limits, `ht[1]` is allocated at double capacity.
- **Rehash Index Tracker:** Commands (e.g., `add`, `find`, `delete`) migrate buckets step-by-step from `ht[0]` to `ht[1]`, tracking progress via `rehashidx` to avoid CPU blocks.

[M2.3] Ziplist & Listpack Memory Alignment

- **Contiguous Layout:** Ziplists store elements sequentially in a single memory block (`[prev_entry_len][encoding][entry_data]`) to avoid pointer memory overheads.
- **Cascade Update Prevention:** Newer versions use Listpacks. Listpacks store element lengths as suffixes rather than prefixes, preventing cascade updates during insertions.

[M2.4] Diet & Skiplist Integration

- **ZSET Dual Layout:** Sorted sets (ZSET) integrate a hash table (`dict`) and a skiplist (`zskiplist`) to support different query patterns.
- **Range Scan Acceleration:** The skiplist supports $O(\log N)$ range searches and sorting, while the hash table provides $O(1)$ key-to-score lookups.

[M2.5] Quicklist & Intset

- **Pointer Optimization:** A `quicklist` is a double-linked list where each node is a compact `ziplist`, reducing memory fragmentation.
- **Integer Array Compression:** An `intset` stores integers in a compact sorted array, automatically upgrading elements (`int16` → `int32` → `int64`) to save memory.

[M3.1] RDB Snapshotting & Copy-on-Write

- **Page Table Sharing:** Executing `BGSAVE` forks a child process sharing the parent's page tables to write database state to an RDB file.
- **COW Page Split:** When the parent process writes to a page, the OS triggers a copy-on-write fault to clone the page, keeping the child's snapshot consistent.

[M3.2] AOF Logging & Fsync Policies

- **Log Buffer Serialization:** Database writes append RESP commands to an `aof_buf` buffer before flushing to disk.
- **Fsync Configurations:**
 - always:** Syncs on every event loop iteration; highly secure but bottlenecked by disk speed.
 - everysec:** Async background thread syncs every second; balances write speed and safety (Standard).
 - no:** Relies on OS write flushes; prone to data loss.

[M3.3] AOF BGREWRITEAOF Rewrite

- **Size Reduction:** When AOF files exceed limits, `BGREWRITEAOF` creates a child process to write the current memory state into a new file.
- **Rewrite Buffer Tracking:** Active writes are appended to an `aof_rewrite_buf` buffer, which is merged into the new file by the main thread after rewriting.

[M3.4] Hybrid Persistence Recovery

- **RDB & AOF Merging:** Hybrid persistence writes the current dataset as a binary RDB snapshot at the header of the AOF file, appending subsequent writes as RESP commands.
- **Recovery Speed:** Bypasses parsing overhead during initial startup phases while retaining incremental write logs.

[🔗] Redis Concurrency & Persistence Trade-offs & Fallacies Matrix

Large single instances are easier to manage and scale than sharded setups: Instances exceeding 32GB experience latency spikes during RDB forks due to page table copying. Network synchronizations can saturate link bandwidth. → Limit single instance footprints to 4GB-8GB. Use Redis Cluster Gossip slots or proxy sharding to scale out, and offload socket processing using Threaded I/O.

Using simple expire commands is a perfect way to reclaim memory for cold keys: Deleting millions of keys at the exact same second stalls the event loop during passive cleanup scans, resulting in latency spikes. → Inject random time jitter to expire values to smooth deletions across time, and rely on Master delete broadcasts to maintain replica consistency.

Everysec AOF logging provides reliable transaction safety with zero CPU cost: disk write latency spikes, the main thread blocking on `write` system calls waits for previous `fsync` operations to finish, stall-locking database access. → Enable `no-appendfsync-on-rewrite` to suspend disk writes during background AOF rewrites, reducing write conflict latency under high disk load.

